

$\gamma_e (\text{And})$

$$f = a.b = \overline{\overline{a.b}} = (\overline{a.b})'$$

Nandın
değili
pibi
düşünebiliriz

Bunu şöyle de
yazabiliriz

$$\hookrightarrow (\overline{a+b})$$

$$(\overline{x+y})$$

$\hookrightarrow \text{Nor}$

Nand

$\nearrow \text{Not}$

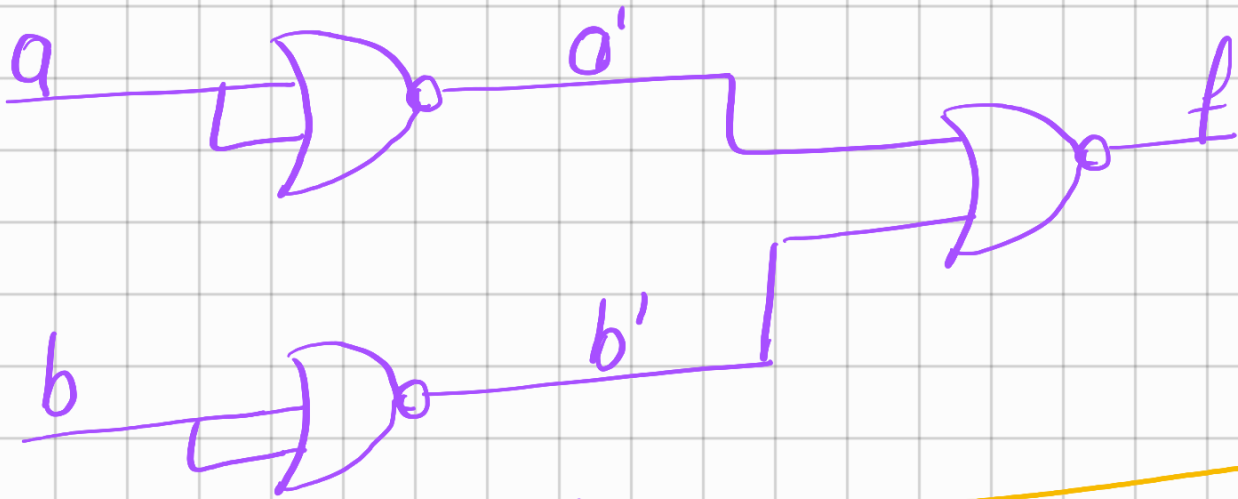


$\hookrightarrow a.b$

↳ elimizde 2 tane Nand kapısı varsa
devreyi Nandlerle yapabiliriz

↳ Bu iki devre de
aynıdır

NoR



↳ $a.b$

Veya (OR)

$$f = (a+b) \rightarrow \overline{\overline{a+b}}$$

$\overline{a+b}$

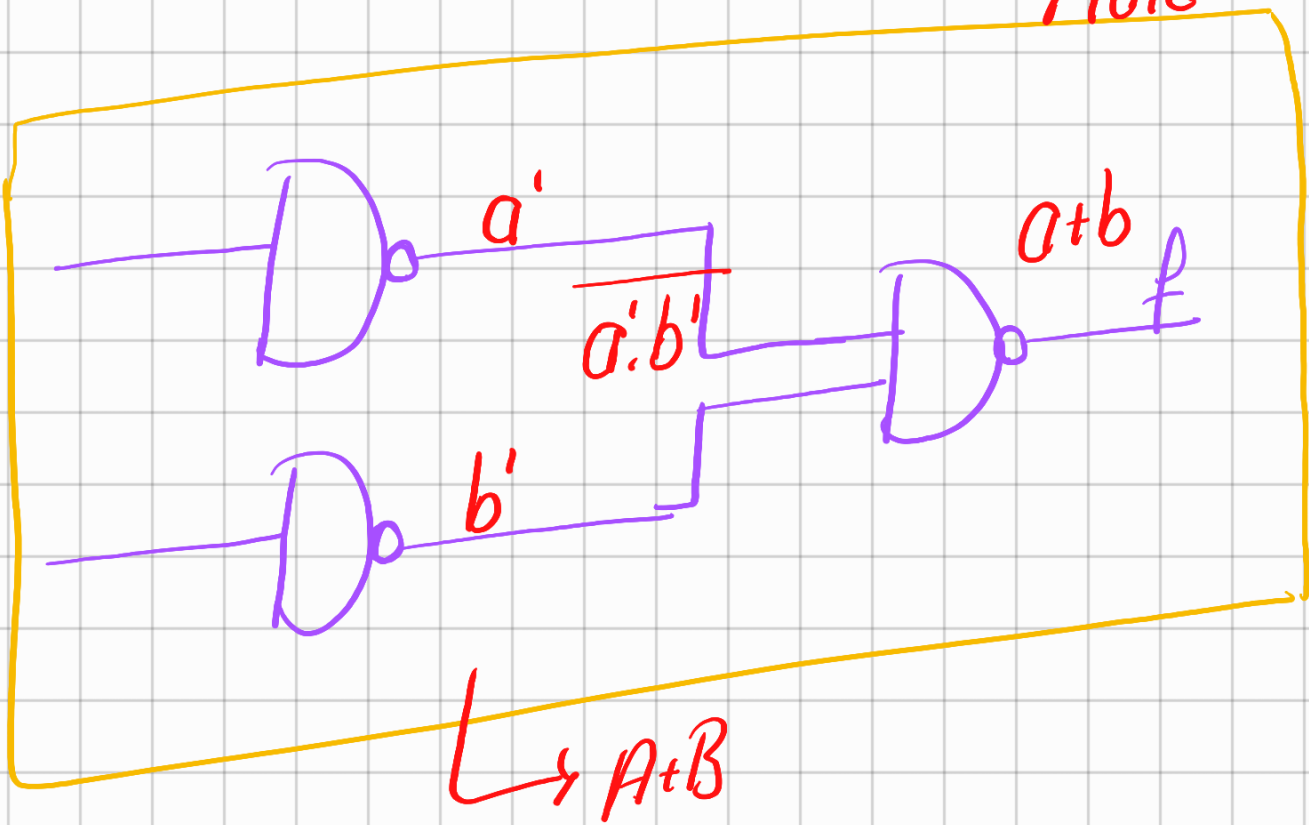
NoR

$a+b$



$\hookrightarrow a+b$

Mod



$\hookrightarrow$ Bu iki derrede birbirinin aynısıdır

$$L = (\overline{a+b})'$$

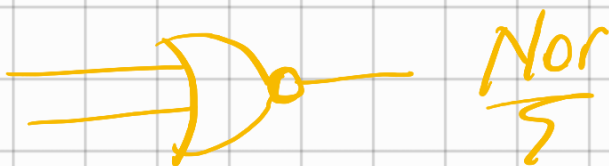
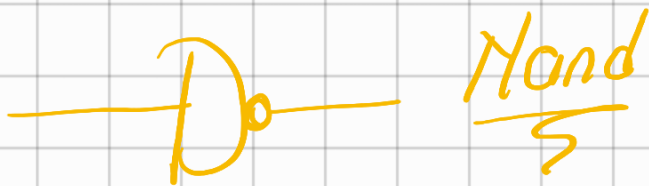
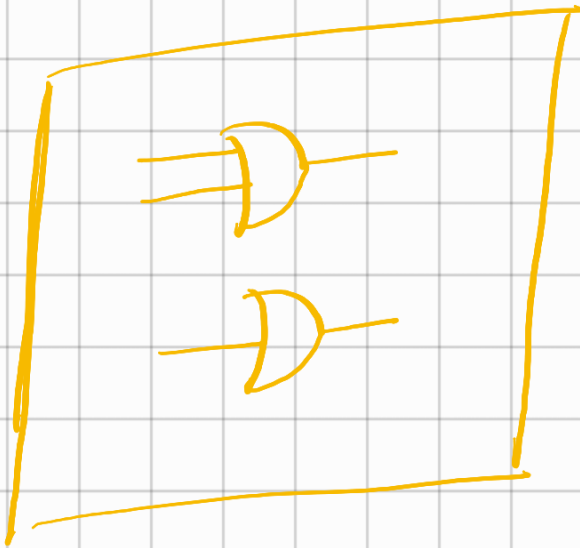
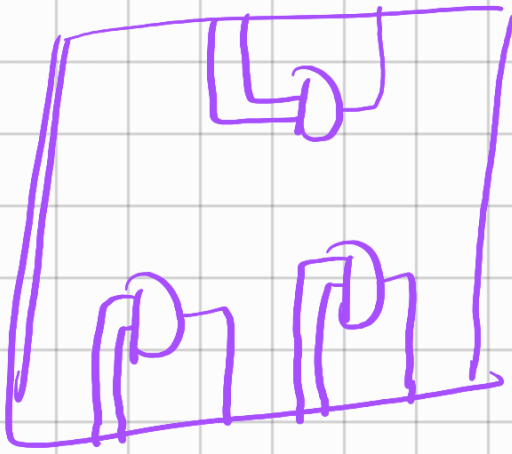
$$= (a'b')$$

$$= (\overline{x.y})$$

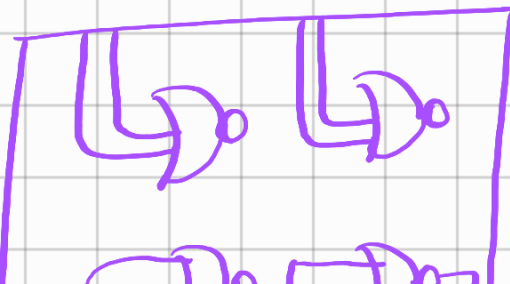
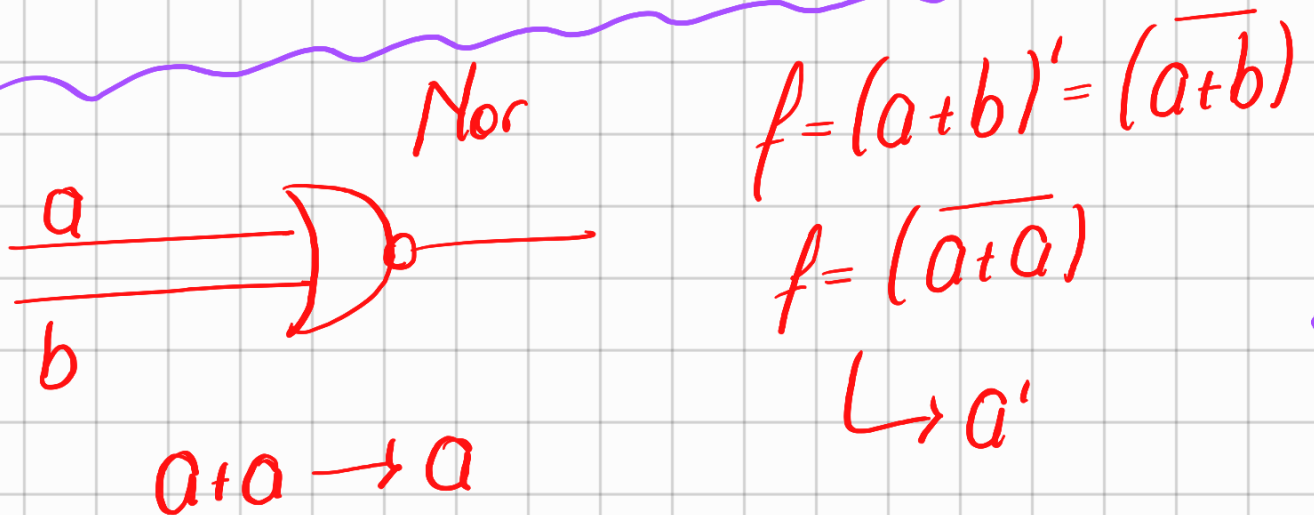
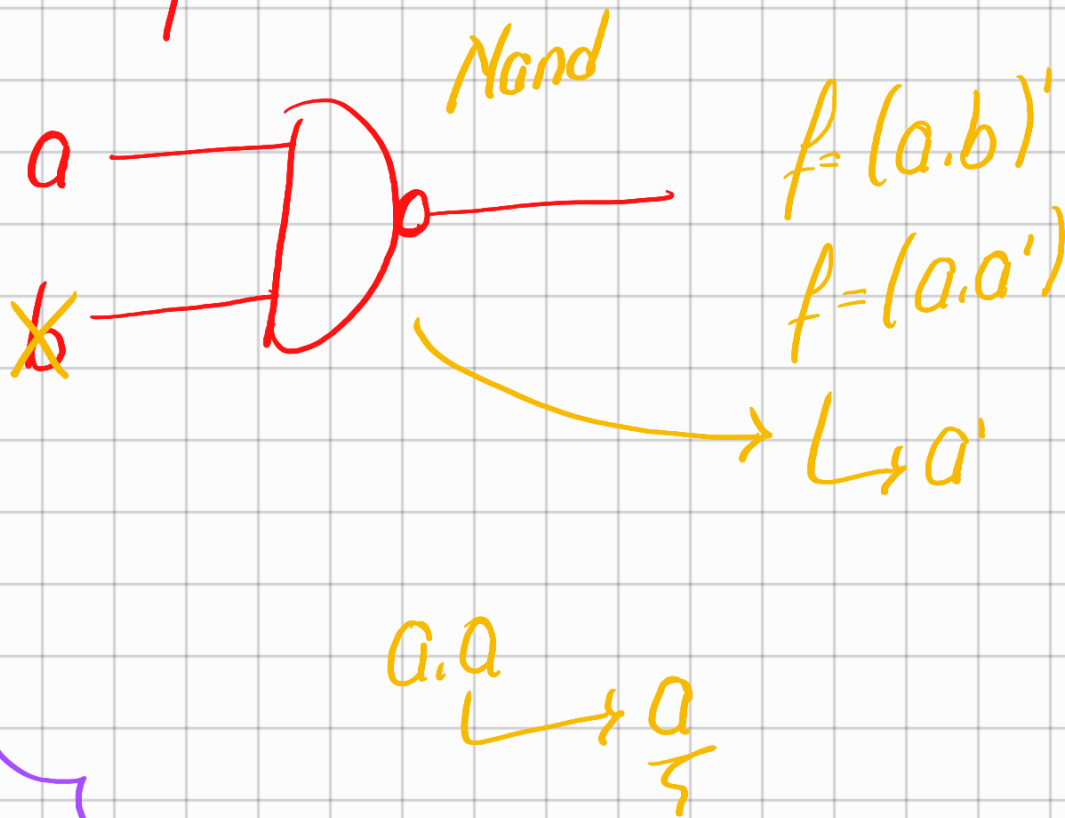
$\hookrightarrow$ Mod

Türetilmiş kapılarla temel lojik kapıların yapılışı

→ Olabildikince aynı kapıları kullanmak
bizim için avantajlı halde



Tümleme Kapısı (Not)





↳ Mesela böyle bir devremiştir
ve Notlarla işlem yapıyoruz
ama bize bir tane de Not lazım
o zaman kapılardan bir tanesini
kısa devre yapıp not kapısına
gevire biliriz

İndirgenmiş ifadelerin aynı tür
kapılarla gerçekleştirilmesi

Garpımların
toplamı

$$f \rightarrow abc + b'c + cd$$

1. maddesi

↳ indirgenmiş
ifadedeki
Garpım şeklindeki
ifadelerimiz

↳ Garpımlar toplamının
en genel ifadesi
şöyledir

$$x \rightarrow a, b, c$$

$$x, y, z$$

$$\rightarrow x_1 x_2 x_3$$

$$f = \sum \pi x_i$$

↳ π_i dedigimiz
şey bunların
Garpımları-
nı
gösterir

↳ Buda toplamadır

Topamlar Garpımı

$$f = (a+b)(b'+c+a)(c+d')$$

↳ Bunlarda toplam şeklindeki ifadelerimiz

$$f \rightarrow \prod \sum x_i$$

Çarpımlar toplamı

TVE (Nand) tasarımı

$$\hookrightarrow \overline{f} = \sum \pi x_i$$

$$(ab + cd)$$

$$(\overline{a}\overline{b})(\overline{c}\overline{d})$$

Burdaki
toplama
çarpmaya
döndü

$$\longrightarrow \overline{f} = \prod \pi x_i \longrightarrow \overline{\overline{f}} = \overline{\prod \pi x_i}$$

Burda da
toplama
çarpmaya
döndü

$$(\overline{a}\overline{b})(\overline{c}\overline{d})$$

Nand (x y)

$$f = \overline{\pi \pi x_i}$$

↳ işlem sonucunda Nandler ile dönüştür-
bilceğin yapıları dönüştürülecek

ÖR $\rightarrow \bar{a}\bar{b}c + \bar{a}bc + a\bar{b}\bar{c} + abc$ Nandler-
le
gerçek-
leyiniz

a	b	c	d	f
0	0	0	0	
1	1	1	1	
1	1	1	1	
1	1	1	1	
1	1	1	1	

a \ bc	00	01	11	10
0		1	1	

$\rightarrow a'c$

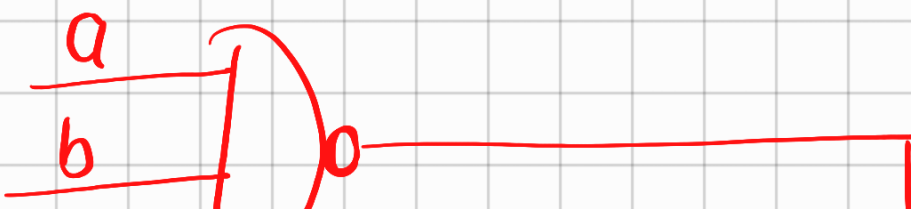


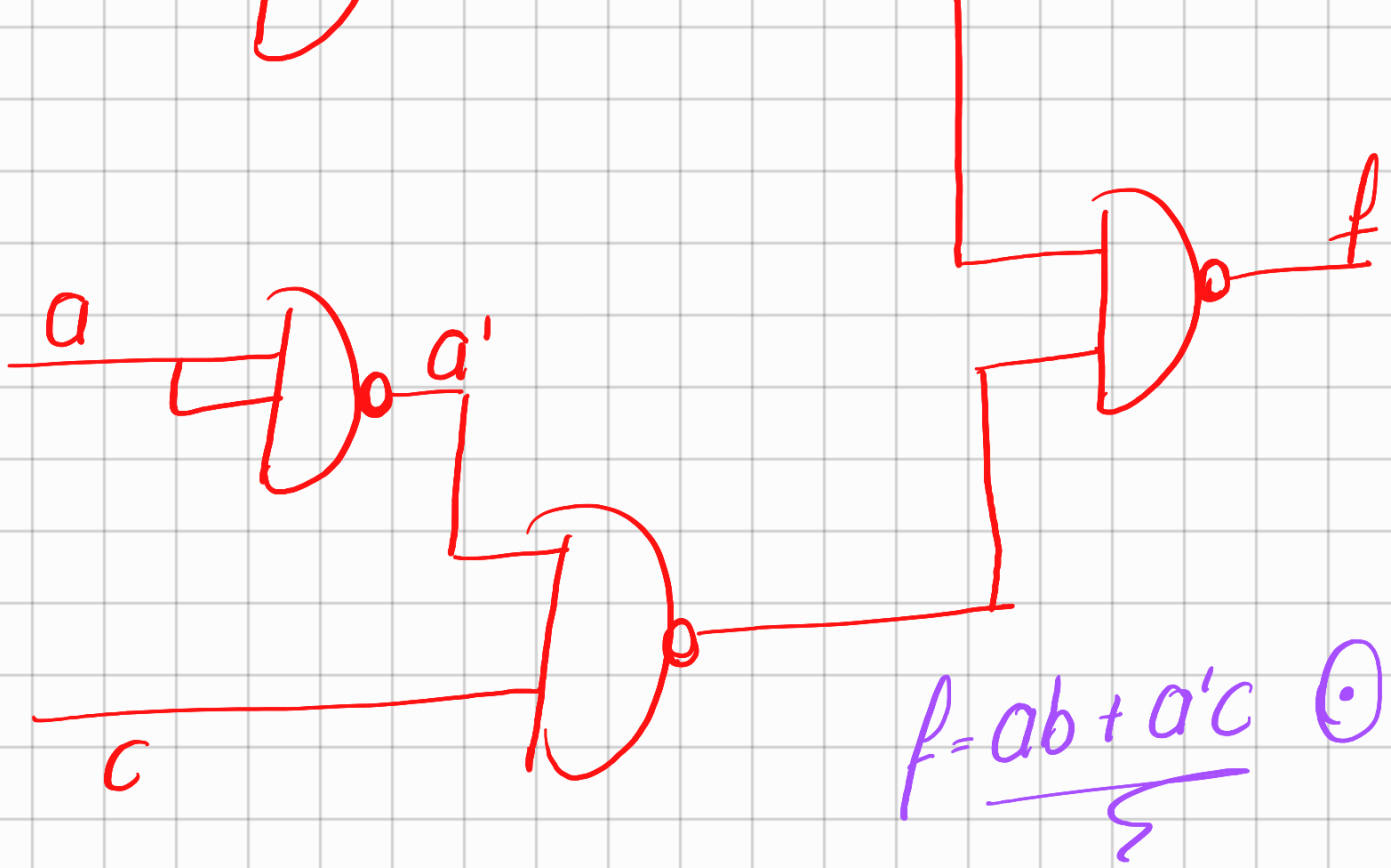
$$\hookrightarrow f = ab + a'c \quad \odot$$

$$\hookrightarrow f = \overline{ab + a'c}$$

$$\hookrightarrow \overline{(ab)'(a'c)'} \quad \begin{matrix} \textcircled{x} & \textcircled{y} \\ \downarrow & \downarrow \\ (x,y)' \end{matrix}$$

$\hookrightarrow$ Çarpımlar toplamı şeklindeki ifadeyi 2 kere değini alınca zaten Nandler ile yapılır hale geliyor





Görpümler toplamı (T veya) NoR
tađarımı
(a+b)'

$$f = \sum \pi x_i \rightarrow \pi x_i = \sum \bar{x}_i$$

$$(ab) \rightarrow \overline{(ab)} \rightarrow (\overline{a+b'})$$

ifadeyi
bu hale
getirmeye
çalışcaz

$$\sum \overline{\sum \overline{x_i}} = f$$

$$\hookrightarrow \overline{f} = \sum \overline{\sum \overline{x_i}}$$

$$\downarrow \overline{f} = f$$

istedigimiz f
fonksiyonunu
elde etmiş
oluyoruz

$$\text{ÖR} \rightarrow f(abc) \rightarrow ab + a'c$$

$$\hookrightarrow (a' + b') + (\overline{a' + c})$$

$$\hookrightarrow (a' + b') + (\overline{a + c}) = f$$

$$(\overline{x + y}) + (\overline{x + y})$$

$$\overline{(\overline{x + y})}$$

$$f = (x+y)$$

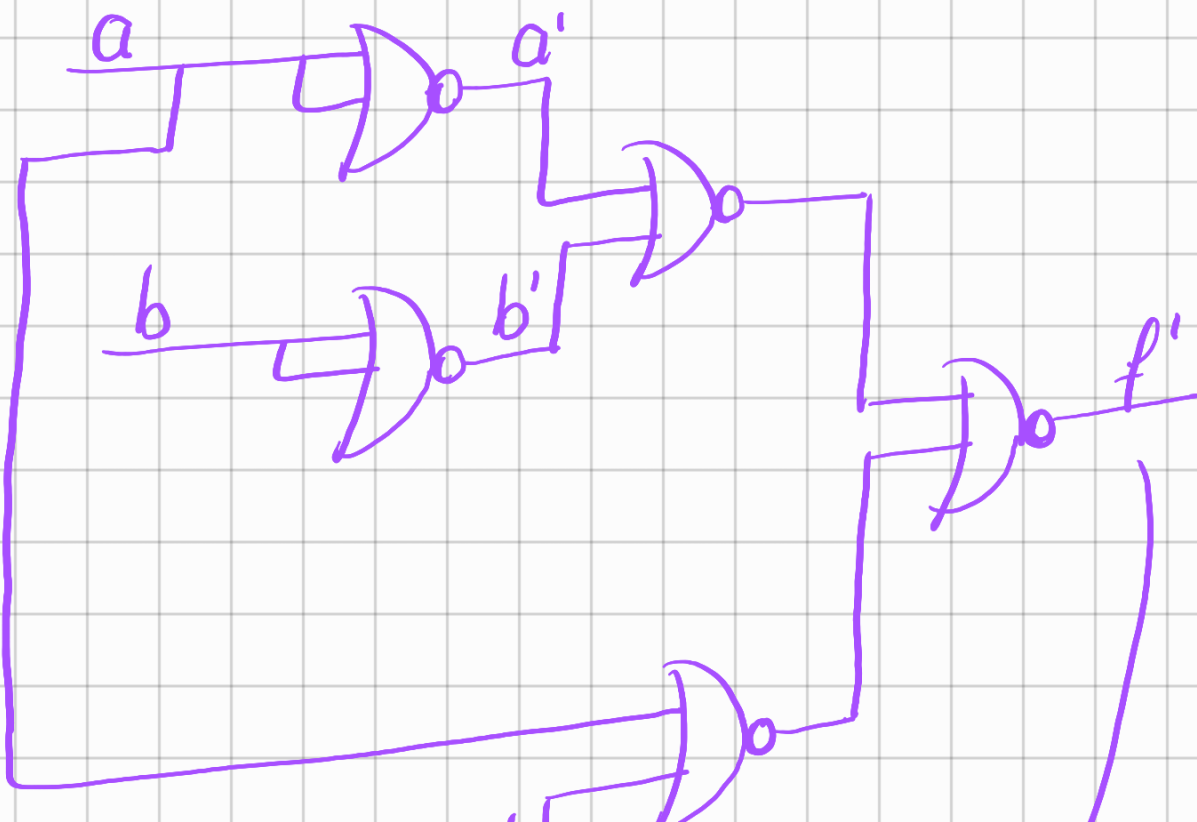
$$f' = (\overline{a'+b'}) + (\overline{a+c'})$$

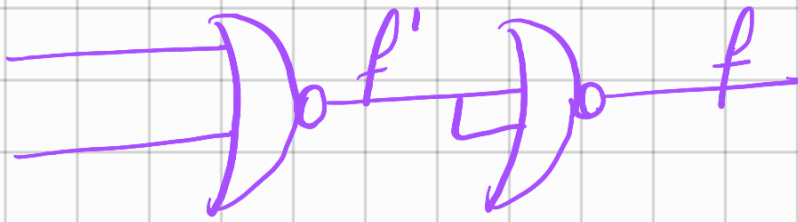
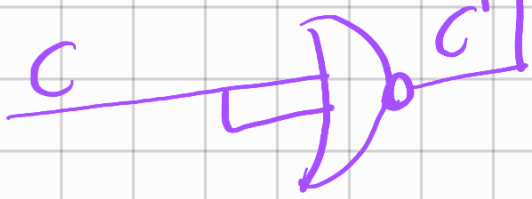
$$(x+y)$$

→ Bunun Nor formunda olmadığını yapmak için



↑
Nor





Toplamlar Çarpımı (Nand) tasarımı

t ve
(tümleyen ve)

$(a.b)'$
 $(x.y)'$

$$f = \prod \sum x_i$$

$$\sum x_i = \overline{\prod \overline{x_i}}$$

$$\overline{(a+b)} = (a'.b')$$

$$f = \prod \prod \overline{x_i}$$

$$\rightarrow \overline{(x_1 x_2)}$$

$$\hookrightarrow (a, b)$$

Non

$$\hookrightarrow \overline{f} = \prod \prod \overline{\overline{x_i}}$$

$$\hookrightarrow f' = \prod \prod \overline{\overline{x_i}}$$

$$\text{ÖR} \rightarrow f(abc) \rightarrow [a+b][\bar{a}+c]$$

$\hookrightarrow$ Bize burda
bunu Nondeyiniz
diyor

$$\hookrightarrow (x, y)'$$

hiç

Aslında biz
bu hale
getirmeye
çalışıyoruz

$$\overline{[a+b]} = (\overline{a' \cdot b'})$$

$$\hookrightarrow (\overline{a' \cdot b'}) (\overline{a' + c'})$$

$$\hookrightarrow f$$

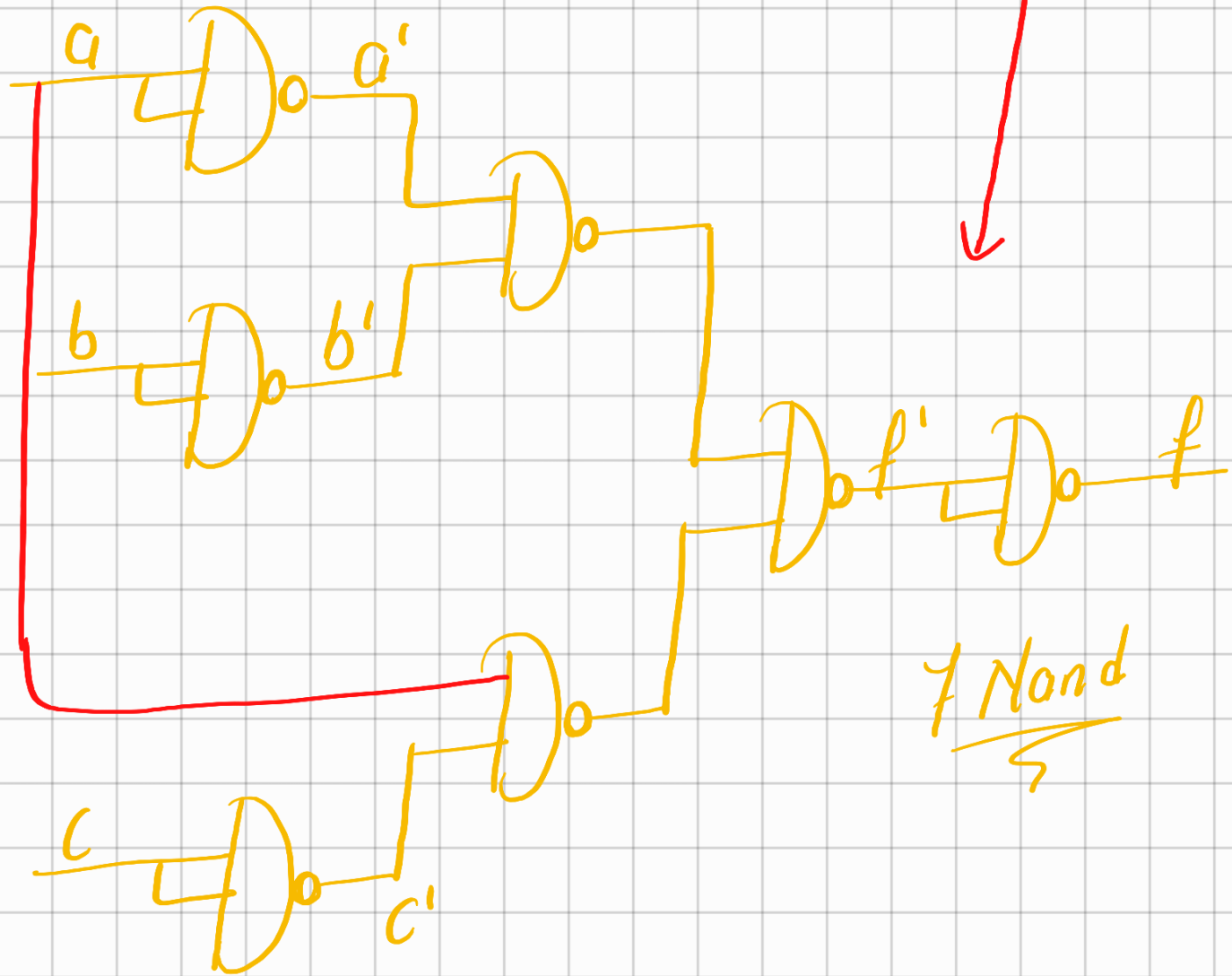
$$\hookrightarrow f = (\overline{a' \cdot b'}) (\overline{a + c'})$$

$\underbrace{\quad \quad \quad}_{X \cdot Y}$

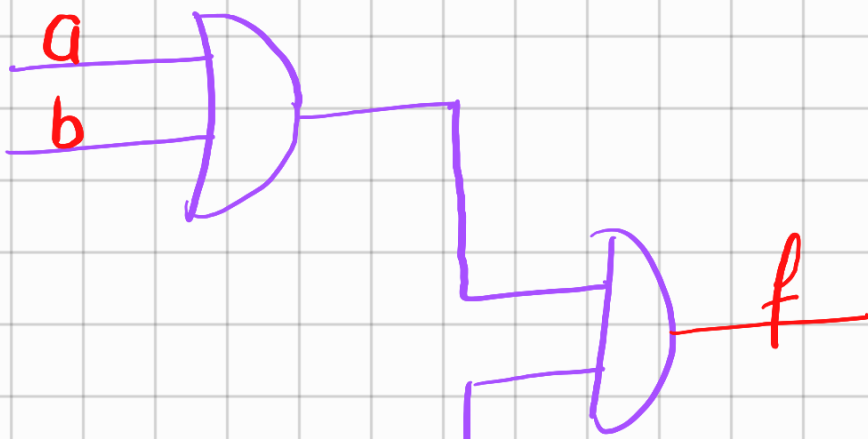
Bakıyoruz burada
normal bir
and işlemi
bunu dönüştür-
mek için
değerini alma-
mız
gerekir
yani
(x.y)'

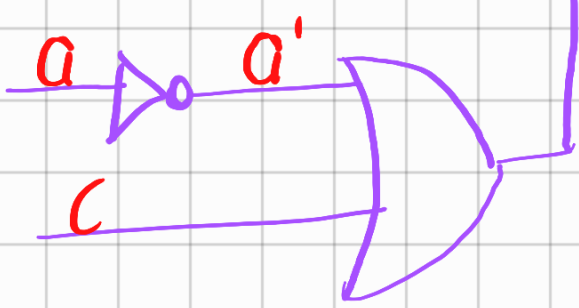
(x.y)

$$f' = (\overline{a'b'}) (a+c')$$



7 Nand



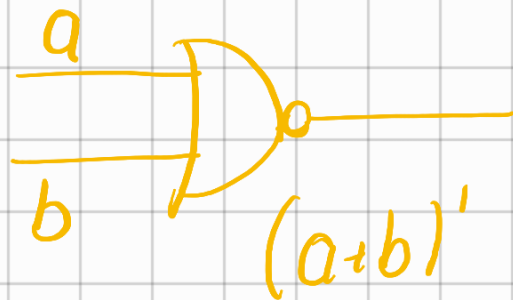


→ eğer bu işlemleri yapmazaydık 3 değişik kapı kullanıcaktık farklı kapılar kullanmak çok kötü bir şey onun yerine bu işlemleri 7 Nord ile hallettik

Toplamlar Grubu (NOR) tasarımı (Tveya)

$$f = \prod \sum x_i$$

$$f' = \overline{\prod \sum x_i}$$



$$f = \sum \sum x_i$$

$$\overline{(f')} = f = \sum \sum \overline{x_i}$$

↳ Bu toplamın
değerini almak
demek Nor
kullanmak
demek

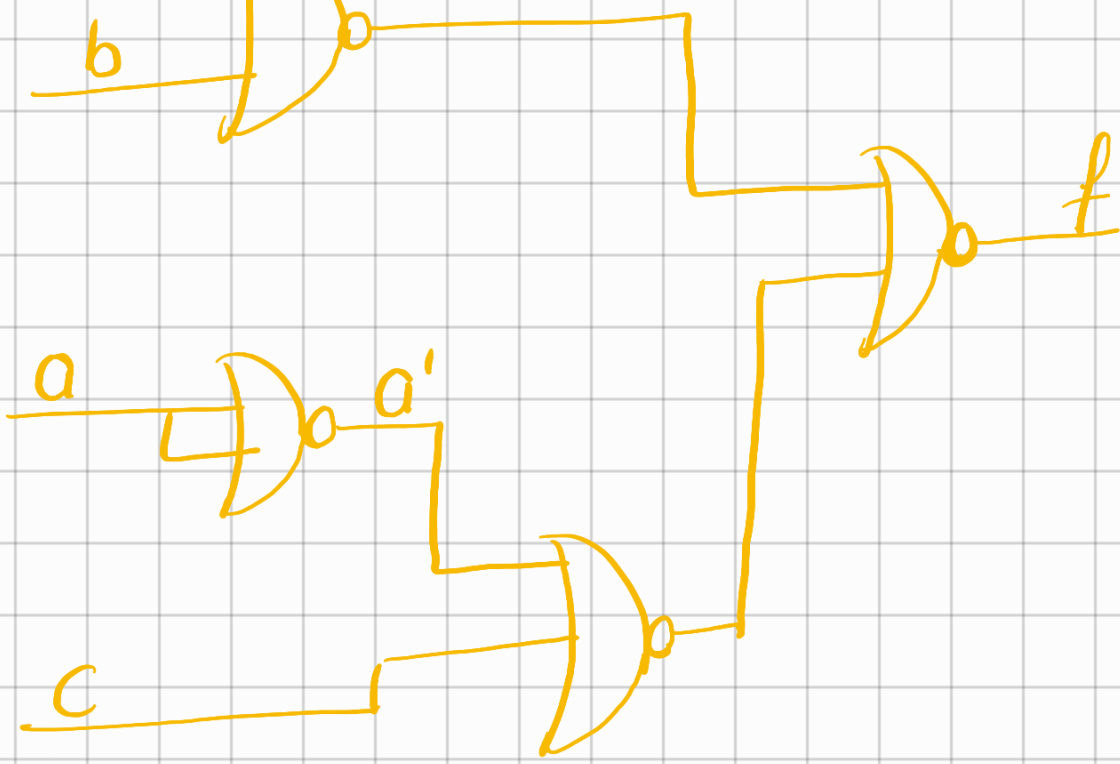
$$\text{ÖR} \rightarrow f(a,b,c) \rightarrow [a+b][a'+c]$$

$$\hookrightarrow f' = \overline{[a+b]} + \overline{[a'+c]}$$

$$f' = x + y$$

$$\overline{(f')} = \overline{\underbrace{[a+b]}_x + \underbrace{[a'+c]}_y}$$

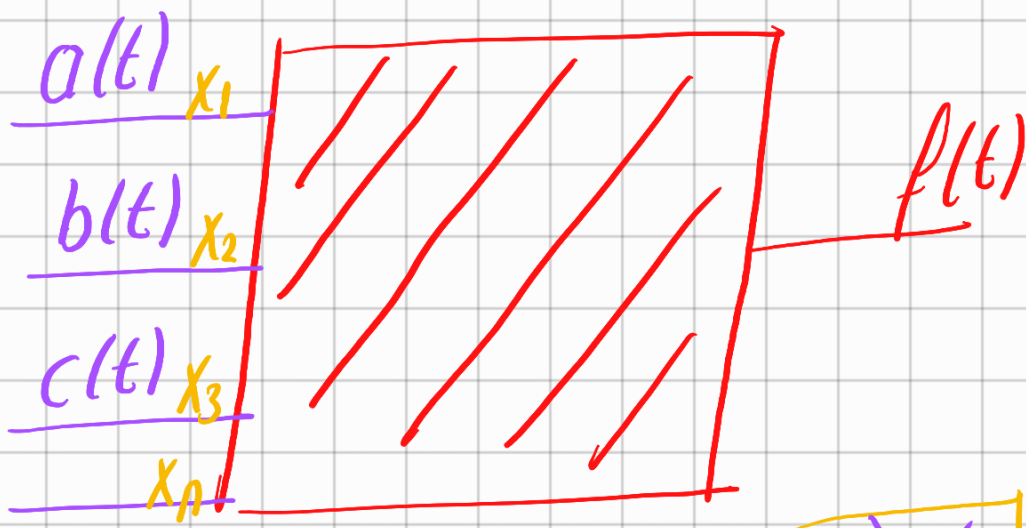
a



Kombinezonel Devreler

↳ Çıkışı sadece o anki giriş degerlerine bağlı olan ve bilinen kapıların biraraya ya getirilmesiyle oluşan lojik devreler

↳ Burada temel kapıları ve türetilmiş kapıları kullanacağız



→ Çıkış bunların t anındaki girişlerine bağlı

→ hafıza bilgisine yapıya gerek yok

→ Çıkış anında t anındaki girişlerden t anındaki çıkış üretiliyor

→ Aritmetik toplama / çıkarma derreleri

yeri toplayıcı / çıkıcı

kontrol / çıkıcı

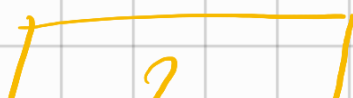
tam toplama

- ↳ Multiplexer (Seçiciler)
- ↳ Demultiplexer (Dağıtıcılar)
- ↳ Decoders (Kod Gözütücü)
- ↳ encoders (Kodlayıcı)
- ↳ ALU (Arithmetic logic Unit)
- ↳ 7 segment display

Kombine Zondal
Derre tasarımı

① → problemin çöz ile tanımı yapılır

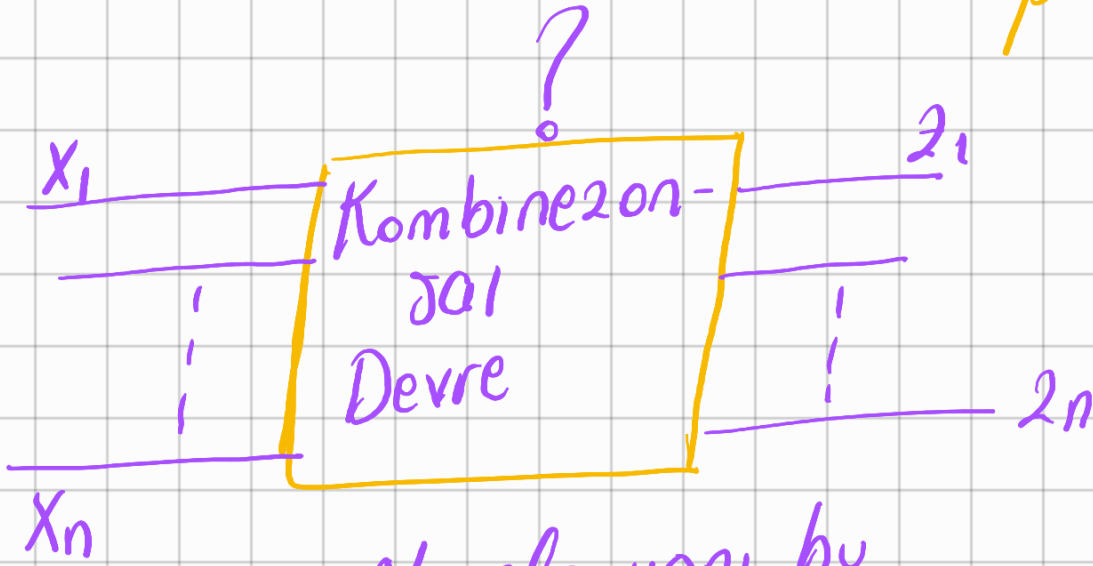
Mezela



Biz bu kutunun
ne iş yapma-
ıştırıyoruz



İmi idiy-
cak input ali-
cak cak out-
put alıcak



Mezela yarı bu
kombinezon-sal devre
ne yapıcak önce
bunun tanımını
yapılmalı

② → giriş ve çıkış değişkenlerinin sa-
yı / isimlendirilmesi

③ → Probleme ilişkin doğruluk tablosu

④ → her bir çıkışa ait lojik fonk mini-
mum maliyet olması için indirge-
nir / pözüme dayalı

(K-Map
Quene-McClusky)

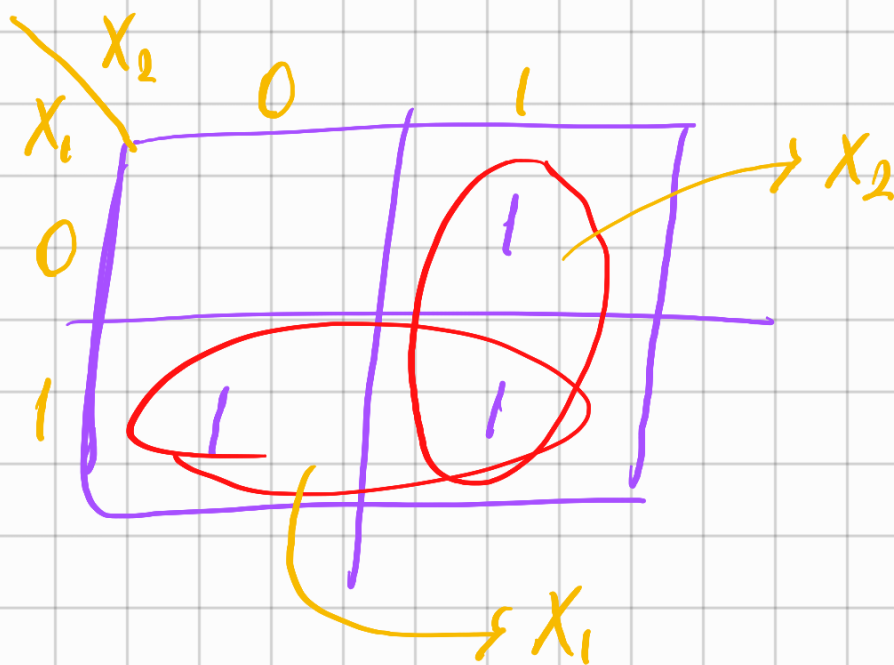
⑤ → (Kullanılcal lojik devre elemanları belirleniyor) tek tip kapı kullanımı istenmiyorsa dönüşümler gerçekleştirilir

⑥ → Devreye ilişkin devre teması çizilir



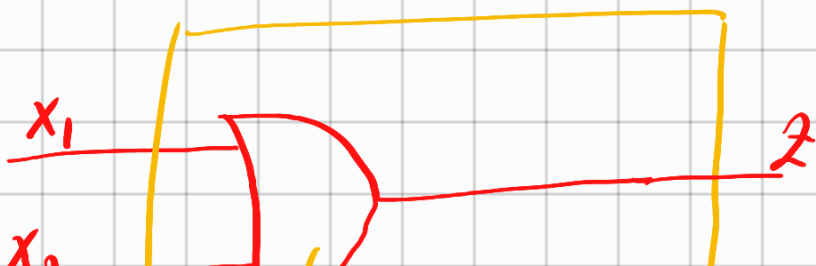
↳ iki girişli bir çıkışlı bir kombinezansal devre olsun ve bu kombinezansal devre de girişlerin ikisi de 0 ise çıkışta lojik 0 aksi durumda devrenin çıkışında lojik 1 olması isteniyor

x_1	x_2	z
0	0	0
0	1	1
1	0	1
1	1	1



$$f \rightarrow x_1 + x_2$$

OR



OR



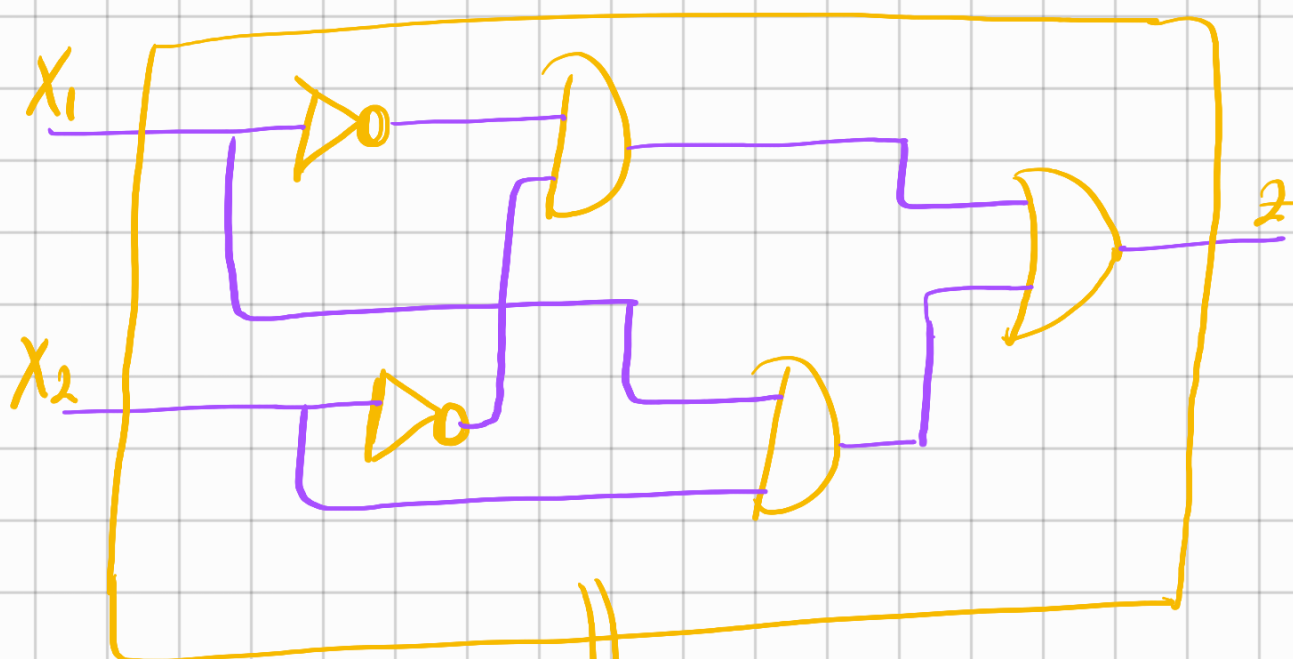
X_1	X_2	Z
0	0	1
0	1	0
1	0	0
1	1	1

→ girişler birbirine eşitse 1 onun haricindeki yerlerde 0 verisin

X_1	X_2	
	0	1

0	1
1	1

$$L \rightarrow f \rightarrow X_1'X_2' + X_1X_2$$



Aynıdır

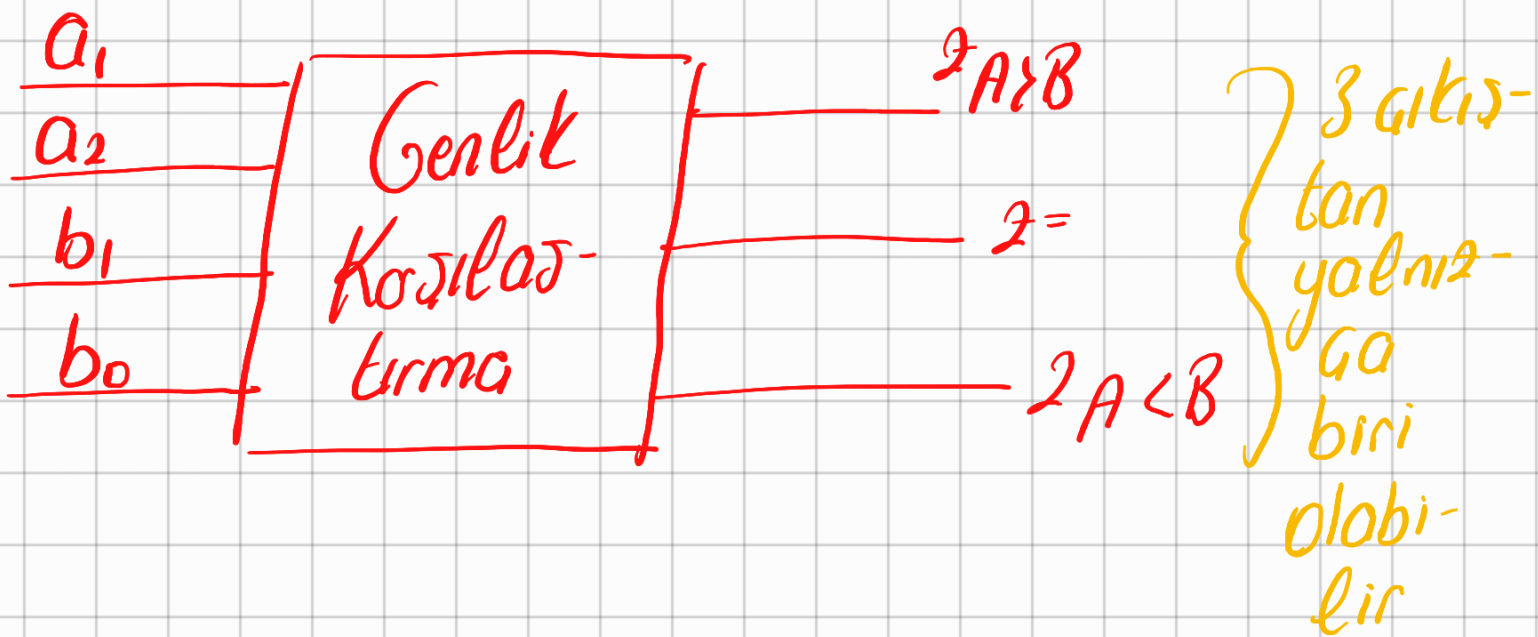


XNOR

4 2 1 10.6 2 2000 01000

OR $\rightarrow$ 2 bitlik 2 sayı
MSB LSB
 $a_1 a_0$ $b_1 b_0$

$\hookrightarrow$ 2 bitlik genlik karşılaştırma
(Amplitude Comparater)



girişler				çıkışlar			
a	a_1	a_0	b_1	b_0	$a > b$	$a = b$	$a < b$
	0	0	0	0	0	1	0
	0	0	0	1	0	0	1
	0	0	1	0	0	0	1
	0	0	1	1	0	0	1
	0	1	0	0	1	0	0

0	1	0	1	0	1	0	0
0	1	1	0	0	0	1	1
0	1	1	1	0	0	1	1
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	0
1	0	1	0	0	1	0	0
1	0	1	1	0	0	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	0
1	1	1	0	1	0	0	0
1	1	1	1	0	1	0	0

16 input

1. Gıkış

2. Gıkış

3. Gıkış

Bunların ayrı ayrı karno maplerini alarak fonksiyonları oluşturcaz

2 a, b

	$a_1 a_0$				
$b_1 b_0$	00	01	11	10	
00		1	1	1	$a_0 b_1 b_0$
					$a_1 \bar{b}_1$



$$\hookrightarrow f_1 = 2 a_1 b = a_1 \bar{b}_1 + a_0 b_1 b_0' + a_1 a_0 b_0'$$

a_1, a_0		b_1, b_0			
		00	01	11	10
		1			
01			1		
11				1	
10					1

$$\hookrightarrow f_2 = 2 a = b$$

$$\hookrightarrow a_1' a_0' b_1' b_0' + a_1' a_0 b_1' b_0 + a_1 a_0 b_1 b_0$$

$$a, a', b, b'$$

$$2 a < b = f_3$$

$b, b_0 \backslash a, a_0$		∞	01	11	10
		∞			
01		1			
11		1	1	1	
10		1	1		

$a', a_0 b_0$ (points to the first column of 1s)
 a', b_1 (points to the first two rows of 1s)
 $b, b_0 a_0'$ (points to the last column of 1s)

$$\hookrightarrow 2 a < b \rightarrow a' b_1 + a' a_0' b_0 + a_0' b_1 b_0$$

a_1		$f_1 \rightarrow 2 a < b$
a_0		

